

How does java script execute code + call

Java script is a single-threaded, synchronous language. This means it can only do one thing at a time, in a specific order. It manages this execution using two main components: the Execution Context and the call stack.

1. The Execution Context:

Before any code runs, the JS engine creates an environment called the Execution Context. Think of it as a wrapper that contains the code currently being evaluated.

There are two main types:

- Global Execution Context (GEC): Created by default when your script starts. It sets up the global object (window in browsers) and the this keyword.

- Function Execution Context (FEC): Created every single time a function is called. Each function gets its own context.

Note :- There is a third one also but not common called Eval Execution Context.

The Two - Phase Pass

Every execution context goes through two distinct phases:

- ① Creation Phase (Hoisting): The engine scans the code. It sets up memory space for variables and functions. Functions are stored entirely, while variables (declared with `var`) are initialized as undefined. (variables with `let` and `const` are placed in a "temporal dead zone").
- ② Execution Phase: The engine reads the code line-by-line (top - to - bottom), assigns initial values to variables, and executes the functions.

The call stack: Keeping Track

The call stack is a mechanism the JS engine uses to keep track of its place in a script that calls multiple functions. It operates on a LIFO (Last in, First out) principle - like a stack of plates. The last plate you put on top is the first one you take off.

→ when the script starts, the Global Execution Context is pushed to the stack.

→ when a function is invoked, a new Function Execution Context is created and pushed to the top of the stack.

→ when that function finishes executing, its context is popped off the stack, and the engine resumes where it left off in the context below it.

Step-by-step walkthrough

```
function multiply(a,b) {  
  return a+b;  
}
```

```
function square(n) {  
  let result = multiply(n,n);  
  return result;  
}
```

```
let output = square(4);  
console.log(output);
```

~~Step~~
Here is exactly how the call stack handles the code execution.

call stack state → what is happening?

Step	
1	[Global] → script starts. Global Execution Context is created and pushed to the stack.
2	[Global] → code reads down to let output = square(4); The engine calls square().
3	[square, Global] → A new context for square(4) is pushed to the top. Execution moves inside square.
4	[square, Global] → Inside square, the engine hits multiply(m).
5	[multiply, square, Global] → A new context for multiply(4,4) is pushed to the top.
6	[square, Global] → multiply calculate is returns the value and its context is popped off the stack.
7	[Global] → square finishes executing returns 16 to output, and its context is popped off.

8 [Global] → console.log(output) prints 16.

9 [] (Empty) → The script finishes. This Global context is popped off. The stack is clear.

* What happens if the stack gets Too Big?

Because the stack has a fixed size. If you call a function that calls itself indefinitely (infinite recursion) without a breaking condition, you will run out of stack space.

```
function chicken() {  
  egg();
```

```
  function egg() {  
    chicken();
```

```
  }  
  chicken();  
}
```

This throws a Range Error: Maximum call stack size exceeded, commonly known as stack overflow.